Modellering af systemet

At modellere et system er en færdighed, der kræver øvelse. Når man skal modellere et system, handler det om at modellere den del af virkeligheden, der skal håndteres i systemet. I opstartsfasen, hvor modelleringen begynder, er det derfor praktisk at have tre områder for øje:

1. Hvilke dele af virkeligheden skal afspejles i systemet?
2. Hvem skal bruge systemet?
3. Til hvad skal systemet anvendes og især hvorfor?

De tre områder repræsenteres af henholdsvis 1) *objektmodellen, 2) aktørerne, som er i kontakt med systemet, og 3) use cases.*

Objektmodellen er en visuel beskrivelse af ting og begreber i den virkelige verden inden for det område, der skal udvikles et system til. Det er her vigtigt at være opmærksom på, at det ikke er objekter i programmet eller tabeller i databasen, men en afspejling af objekter i den virkelighed, som it-systemet skal afspejle.

Det er altså en tegning eller en model, hvor man kan se eksempler på de informationer, der skal behandles i systemet, og hvordan de hænger sammen.

Fremgangsmåde

Hvordan kommer man i gang med at tegne en objektmodel? Det er der mange svar på. Én fremgangsmåde kan være: 1) at starte med at identificere og visualisere stamdata; 2) derefter identificere og visualisere samlende objekter og deres relationer til stamdataene. Derved bliver udformningen af en objektmodel opdelt i to trin i opstartsfasen.

Hvad menes der så med stamdata og samlende objekter? *Stamdata*, er de data som har en lang levetid i systemet og forholdsvis sjældent skifter tilstand. Det er vigtigt at få stamdataene repræsenteret og adskilt i objektmodellen. Når stamdataene er identificeret, adskilt og repræsenteret som objekter i objektmodelskitsen, er det tid til at identificere de samlende objekter. Her skal man spørge sig selv: hvad er det for nogle objekter, der samler de andre objekter i objektmodellen, eller sagt med andre ord; hvilke typer af objekter har relationer til de fleste andre objekter? Det særlige ved disse objekter er, at de som oftest har en relativ kortere levetid end stamobjekterne. Identificeringen af de samlende objekter sker helt naturligt i forbindelse med, at man overvejer relationerne mellem stamdataene og de samlende objekter. Mange begynder ofte for tidligt i processen med at danne relationer mellem objekter, altså før de har overvejet hvilke objekter, der fungerer samlende i systemet.

Lad os tage et eksempel som kan illustrere fremgangsmåden.

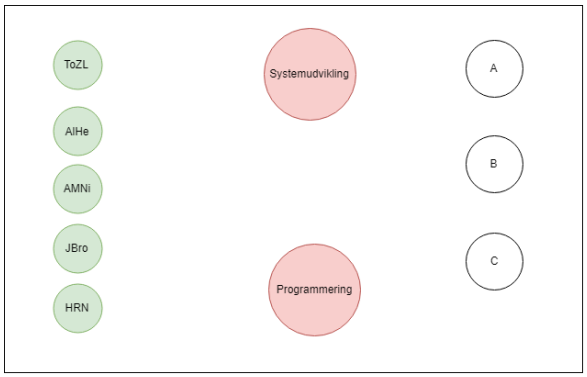
Eksempel – system som viser fagfordeling mellem undervisere

Vi skal lave et system, som viser, hvordan de forskellige fag og hold er fordelt mellem underviserne på datamatikeruddannelsen på første studieår.

Identificering af stamdata

Først ser vi på hvilke stamdata vi kender?

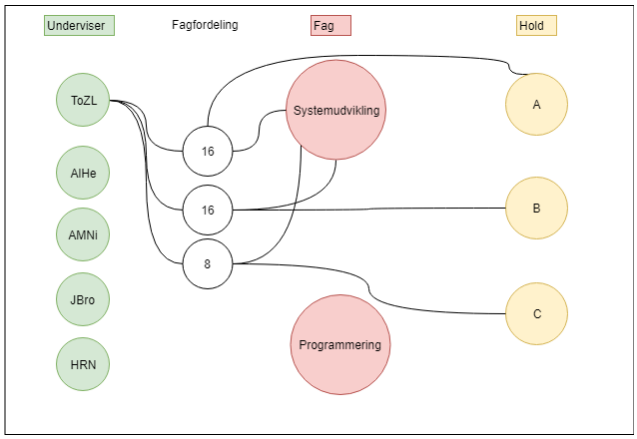
Vi ved at der er en række undervisere på første studieår; Tove, Allan, Asger, Jan, og Hans. Underviserne repræsenterer altså stamdata, så dem tegner vi ind. Vi ved også at der er to fag på første år henholdsvis systemudvikling og programmering, de tegnes ligeledes ind. Slutteligt ved vi at der er tre hold som skal have undervisning, nemlig A, B og C (se figur 1).



Figur 1- Stamdata

Identificering af samlende objekter

Nu har vi fået identificeret og tegnet stamobjekterne i systemet, så skal vi have identificeret de samlende objekter i vores system. Hvordan gør vi det i dette tilfælde? Vi ved at vi skal lave et system, der viser fagfordelingen mellem undervisere. Altså skal vores system vise hvilken underviser, der har hvilket fag med hvilket hold. Dette kan vi repræsentere med et objekt der viser, hvor mange lektioner, der skal dækkes om ugen.



Figur 2 - Samlende objekter

Objekterne med de 8 og 16 lektioner bliver samlende for stamobjekterne i objektmodellen. Tager vi eksempelvis udgangspunkt i Tove, som underviser i 16 lektioner om ugen i systemudvikling på Hold A. I det tilfælde samler de 16 lektioner de andre objekter og vi kan ud fra dette objekt identificere, hvilken underviser der er tale om, hvilket fag der er tale om, hvilket hold der er tale om og i hvor mange lektioner.

Navngivning af klasser

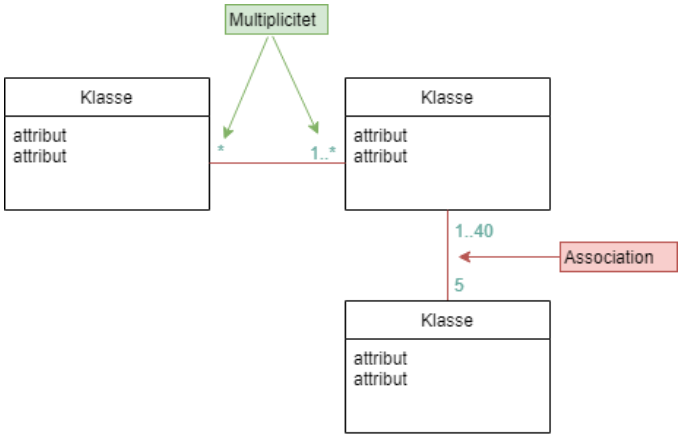
Den skarpe læser vil have opdaget en tilføjelse ud over det samlende objekt i figur 2. Øverst i figur 2 er der tilføjet fire ord; *Underviser, Fagfordeling, Fag og Hold*. Disse fire ord repræsenterer klasserne, som objekterne hører under. Navngivningen af disse klasser er vigtig, da de påvirker resten af udviklingen af systemet og overføres direkte til Domænemodelen. Hvor et objekt er *en helhed med identitet, tilstand og adfærd* (Mathiassen et al. 2001, s. 4)[1] så er en klasse en beskrivelse, som samler objekter med samme struktur, adfærdsmønstre og attributter (Mathiassen et al. 2001, s. 4).

Når man navngiver klasserne, er det vigtigt, at man tager udgangspunkt i det sprog, som bruges inden for den virksomhed, som man laver systemet for. Ser man f.eks. på klassen ”Underviser”, kunne den uerfarne datamatiker have kaldt klassen for ”Lærer”. Dette klassenavn ville være rigtigt på en folkeskole, men da systemet skal laves for UCL er underviser det rigtige begreb, da det er det udtryk, der bruges på UCL.

Navngivningen er med til at skabe sporbarhed gennem hele systemet. Det er derfor vigtigt, at man i analysefasen sikrer sig, at man benytter de korrekte begreber. Dette gør man gennem tæt samarbejde med kunden, der skal godkende systemet.

Domænemodel

Hvor objektmodellen giver et indblik i konkrete data fra domænet, fungerer domænemodelen som en mere generel oversigt over, hvilke slags objekter, der kan være i systemet ved at inddele dem i klasser og attributter. Associationen mellem klasserne vises med linjer mellem klasserne, hvor multipliciteten noteres (se figur 3). Uddybning af multiplicitet findes i Larman (2005) s, 153-156.



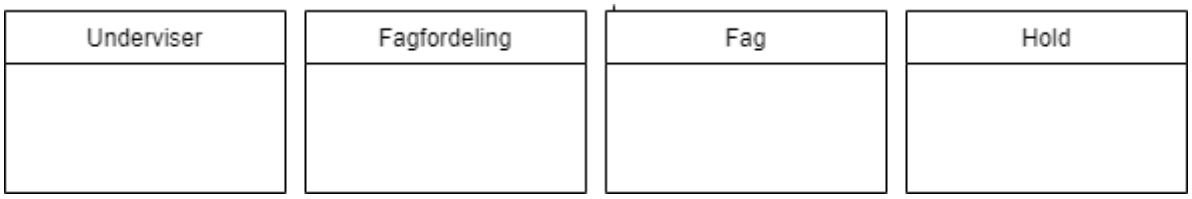
Figur 3 – Domænemodel (klasser, attributter, associationer og multiplicitet)

[1] Mathiassen, L., Munk-Madsen, A., Nielsen, P. A., Stage, J., (2001). Objekt Orienteret Analyse og Design. Forlaget Marko. Aalborg

Domænemodel med udgangspunkt i objektmodel

At tegne en domænemodel er en simpel opgave, hvis man har lavet sit forarbejde ordenligt i form af objektmodellen. Det sværeste i denne proces er at finde passende navne for attributterne. Igen skal navnene tage udgangspunkt i det sprogbrug, der anvendes i virksomheden/domænet (i dette tilfælde UCL).

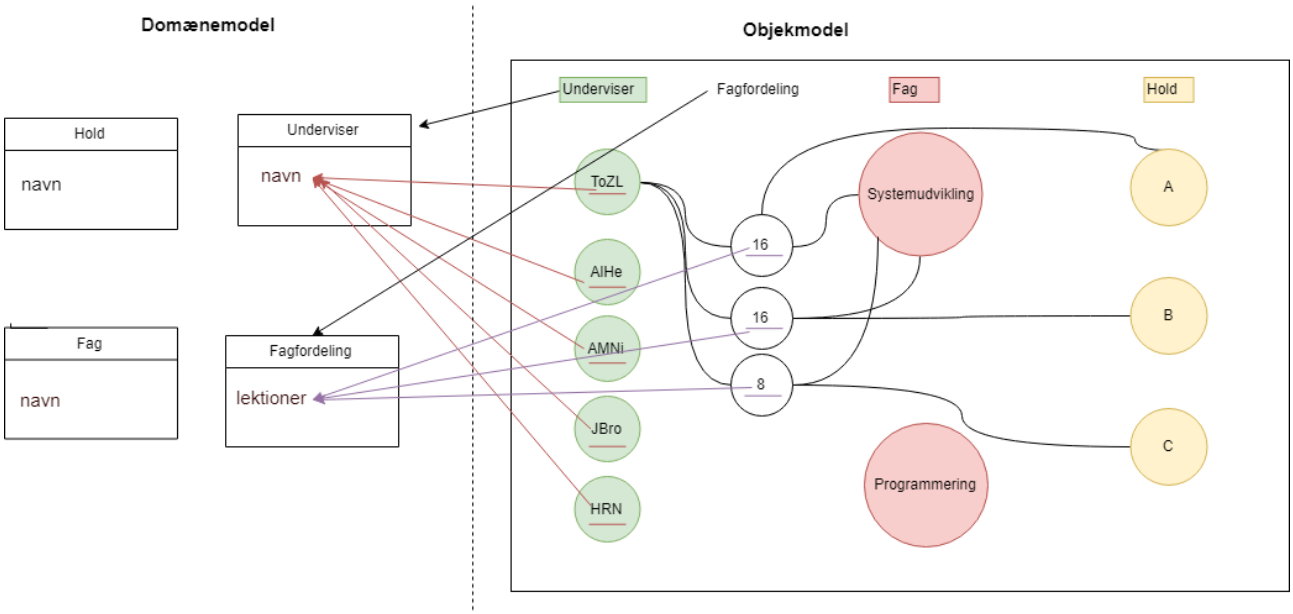
I første omgang er det en ren kopieringsopgave fra objektmodellen. Man tager først udgangspunkt i de klasser, man har fundet i objektmodellen. I tilfældet med fagfordelingssystemet er der tale om ”Underviser”, ”Fagfordeling”, ”Fag” og ”Hold". Bemærk at navnet på en klasse altid skrives med stort begyndelsesbogstav og i ental. Vi skal altså nu tegne fire kasser, med klassenavne øverst (figur 4).



Figur 4 – Klasser fra objektmodelen

Derefter skal vi have udfyldt klassernes attributter. En attribut kan beskrives som en fællesbetegnelse, der dækker over den type data, som findes på den konkrete plads i objektmodellen. I figur 2 er ”programmering” og ”systemudvikling” i de rosa cirkler eksempler på to konkrete data, der dækker over to forskellige fag. Fælles for de to begreber er, at de begge er navne på de konkrete fag. Fællesbetegnelsen bliver derfor ”navn”.

Et andet eksempel i figur 2 er tallene under Fagfordeling 16, 16 og 8, i de hvide cirkler. Disse tre tal dækker alle over det antal lektioner, underviseren har med det enkelte hold. Et sigende navn i domænemodellen vil derfor være lektioner (se figur 5).



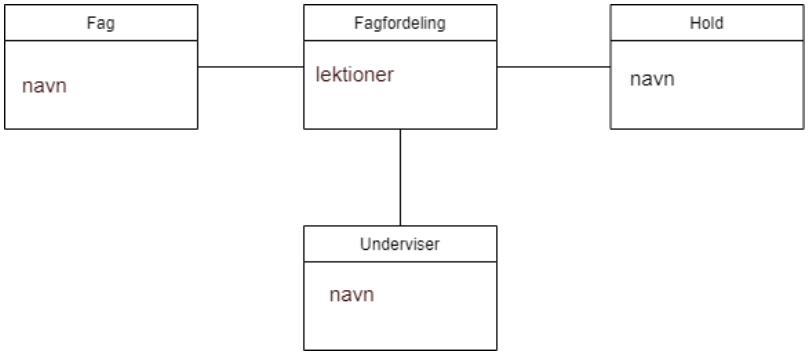
Figur 5 – Navngivning af attributter

Vi giver ligeledes dataene i ”Hold” og ”Fag” objekterne fællesbetegnelsen navn, som vist i figur 5. Det skal pointeres, at der i dette tilfælde, er der tale om et meget simpelt domæne, og det er helt tilfældigt at ”navn” går igen som attribut i tre af klasserne, dette kan sagtens se anderledes ud i andre domænemodeller. Der er også typisk flere attributter i hver klasse.

Associationer mellem klasserne

At tegne associationer mellem klasserne er også stort set en kopieringsøvelse fra objektmodellen til domænemodellen.

Vi kan aflæse i objektmodellen, at Tove har en række relationer til objekter med tal. Der skal altså være en association mellem klasserne ”Underviser” og ”Fagfordeling”. Der er også associationer mellem systemudvikling og tallene, og der er associationer mellem tallene og holdene. Disse associationer tegnes med en streg mellem klasserne i domænemodellen, som vist i figur 6.



Figur 6 – Associationer

Når vi har tegnet associationerne ind i domænemodellen skal vi have noteret associationernes multiplicitet. Multiplicitet definerer, hvor mange objekter af en klasse, der kan være associerede med et objekt af en anden klasse (Larman, 2005, s, 153).

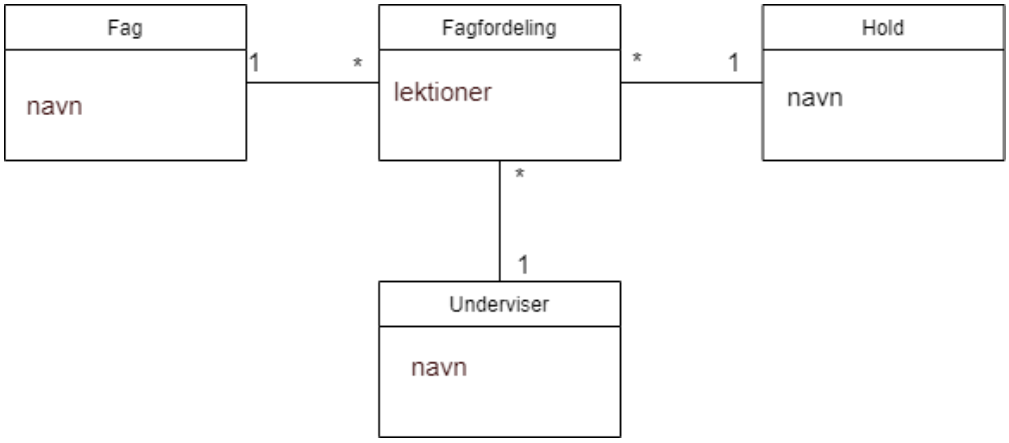
Hvis vi igen tager udgangspunkt i klassen ”Underviser”, og kigger i objektmodellen (figur 2), kan vi se, at objektet Tove har tre streger, som går over til hver deres Fagfordeling-objekt.

Når man kigger på multiplicitet, er det vigtigt at spørge sig selv, om der kunne findes flere eksempler af associationer, som ikke er vist i objektmodellen. Spørgsmålet er altså, om Tove kun kan have relationer til ”tre” af Fagfordelingerne, eller om hun kan have flere, end der er vist i objektmodellen og derved ”mange”. Og vi skal ligeledes spørge, om fagfordelingen kan have relation til en underviser, eller om den kan have relation til flere undervisere.

I dette tilfælde kan en underviser have mange Fagfordelinger, og Fagfordelingen er for den enkelte underviser kan en fagfordeling kun have én underviser. En underviser kan ligeledes eksistere i systemet uden at være tilknyttet til en fagfordeling. Derfor kan en underviser have nul til mange fagfordelinger, hvilket vises med en stjerne (*)

De enkelte fagfordelings-objekter er ligeledes knyttet til et fag, hvilket vil sige at alle 16 eller 8 lektioner enten er systemudvikling eller programmering. Men faget kan godt være tilknyttet til flere fagfordelingsobjekter.

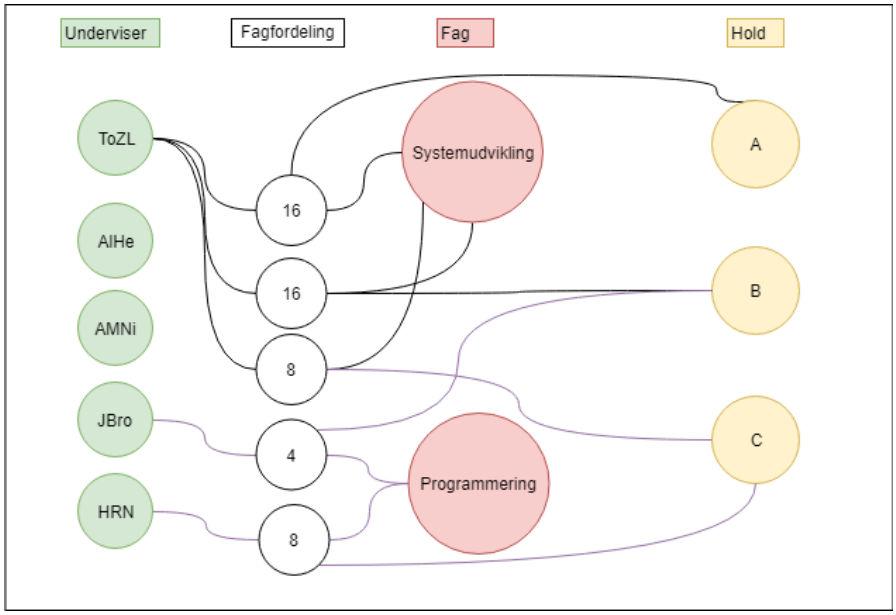
Slutteligt er fagfordelingsobjekterne tilknyttet et bestemt hold, men holdet kan godt være tilknyttet flere forskellige fagfordelings-objekter. Et hold har jo både systemudvikling og programmering.



Figur 7 - Multiplicitet

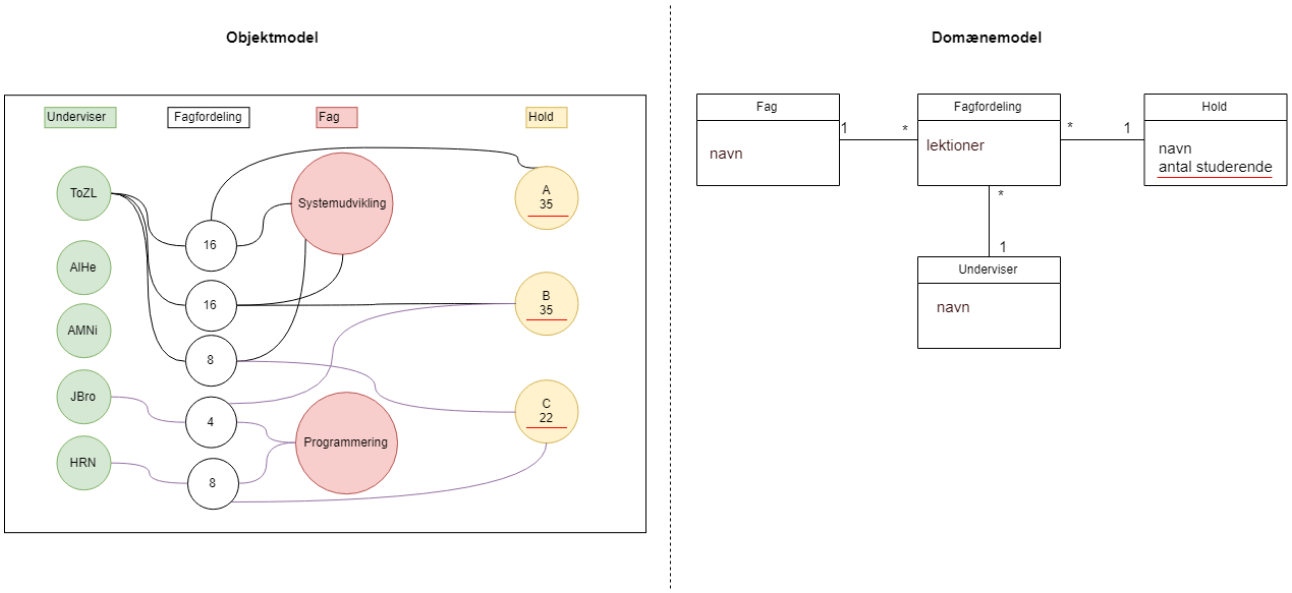
Sporbarhed og kvalitetssikring mellem objekt- og domænemodel

I processen med at skabe domænemodellen har vi nu bemærket, at objektmodellen ikke viste alle de typer af associationer, som vi kom frem til, at der skal være i domænemodellen. Derfor går vi tilbage til objektmodellen og opdatere den, så der er overensstemmelse mellem de to modeller. Objektmodellen skal altid afspejle det, vi viser i domænemodellen og omvendt.



Figur 8 – Objektmodel opdateret

Det samme gælder den anden vej. Hvis vi pludselig opdager, at kunden også gerne vil kunne se hvor mange studerende, der er på hvert hold, skal dette skrives ind i begge modeller. Først tilføjer vi hvert holds konkrete antal studerende i objektmodellen, og efterfølgende opdaterer vi domænemodellen så der er overensstemmelse mellem de to. (se figur 9)



Figur 9 – opdatering af modeller